Lab session 3

<u>Objective of the Lab session:</u>
In this lab session, we will code a classical emulator for emulating the evolution of a quantum analog system. This session aims to emulate the systems manufactured by the D-Wave company, a private company established in Canada that sells quantum annealers.
As the emulation of such a process involves matrix sizes scaling exponentially with the number of qubits involved in the system, the emulator will only be able to solve small systems. You should work with instances smaller than 15 variables.
For example, even with this small number of variables, you will manipulate matrices of sizes:

$$A \in \mathbb{R}^{32768 \times 32768}$$

If your computer is not able to produce results in descent time, do not hesitate to further diminish the size of the input instances (9 or 10 should produce results in descent time).

D-Wave systems are designed to minimize an Ising problem. In the first session, we saw that instances of Ising problems can be represented by an interaction graph where nodes represent the variables s_i with an associated weight w_i and where edges between s_i and s_j can be used to store weights $w_{i,j}$.

For each Ising problem instance, we are able to build an Ising Hamiltonian which encodes H_{final} by replacing each s_i by the tensor product :

$$\sigma_i^z = I_1 \otimes I_2 \otimes ... \otimes I_{i-1} \otimes \sigma_i^z \otimes I_{i+1} \otimes ... \otimes I_n$$

and each quadratic relation $s_i s_j$ by the tensor product :

$$\sigma_i^z \sigma_j^z = I_1 \otimes I_2 \otimes ... \otimes I_{i-1} \otimes \sigma_i^z \otimes I_{i+1} \otimes ... \otimes I_{j-1} \otimes \sigma_j^z \otimes I_{j+1} \otimes ... \otimes I_n$$

with $\sigma^z = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}$

Doing this substitution should give you a square matrix with 2^n entries where n is the number of variables. The initial Hamiltonian used to start D-Wave evolution is fixed and consists of a combination of σ^x operators for each qubit. Each σ^x operator can be implemented by the tensor product :

$$\sigma_i^x = I_1 \otimes I_2 \otimes ... \otimes I_{i-1} \otimes \sigma_i^x \otimes I_{i+1} \otimes ... \otimes I_n$$

with $\sigma^x = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}$
The tensor product can be realized in python using the function kron of numpy.

<u>TODO 1:</u> In the directory quantum_solver, create a new Python file dwave_simulator.py that will store the class DwaveSimulator used for computing the quantum computer's evolution.
The class should have the following attributes :

- annealing_schedule : defines the annealing schedule of functions A(s) and B(s). Assume at first that this schedule in a linear schedule composed of 101 points with:
  $A(s) \in [1, 0.99, 0.98, ..., 0.01, 0]$ and $B(s) \in [0, 0.01, 0.02, ..., 0.98, 0.99, 1]$
  where s is the index in the list.

TODO 2 : Create a method build_Hfinal(ising_problem) in the class DwaveSimulator that takes as input the Ising problem and returns the corresponding matrix Hfinal. You can for example use the sympy expression to iterate over the linear and quadratic terms.

TODO 3: Create a method build_Hinit(n) in the class DwaveSimulator that builds the matrix Hinit from the number of variables n used in the Ising problem.

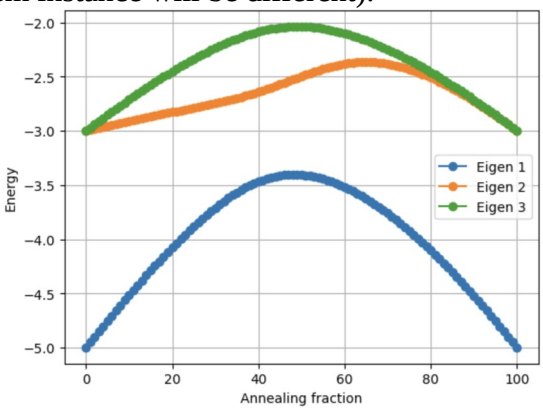
TODO 4: Create a method simulate_evolution(ising_problem, nb_eigenvalues) in the class DwaveSimulator that simulates the D-Wave evolution process. This method should build the matrices H_{init} and H_{final} once, then create and diagonalize the Hamiltonian H(s) for each time step specified in the annealing_schedule considering the equation (consider the linear schedule given in the TODO 1):

$$H(s) = A(s)H_{init} + B(s)H_{final}$$

It is not asked to develop a diagonalization algorithm, reuse existing methods instead (see for example the following tutorial using a scipy method : https://www.sfu.ca/~jtmulhol/py4math/linalg/np-decomp/).
Record the list of eigenvalues (remember, the lowest energy eigenvalues are very important to us as it will later characterize the time required for the quantum evolution).

Return the list of eigenvalues at each time step. The parameter nb_eigenvalues determines how many different eigenvalues are returned.
Check that the lowest eigenvalue of the last time step corresponds to the optimal solution to the Ising problem and that the eigenvector corresponds to the ideal affectation of variables $s = (s_1, s_2, s_3, ..., s_n)$.

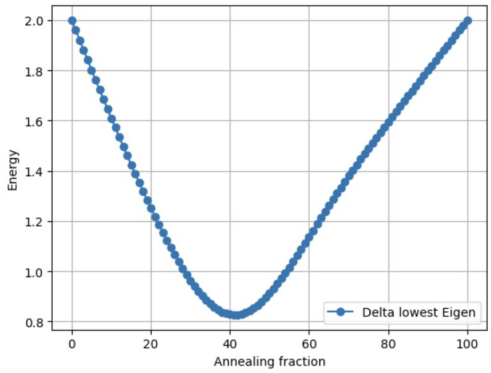
TODO 5:
This todo aims to analyse the spectral gap of problems solved with the quantum annealing algorithm. For this purpose, you will create plot functions that display the different eigenvalues for each annealing time step. You should obtain plots that look like the following one (they will be different because your problem instance will be different).



A second plot will be used to displays the spectral gap of the problem. The spectral gap corresponds to the minimum energy difference between the lowest and second lowest eigenvalue at any fraction of time:

$$g_{\min} = \min_{0 \leq s \leq 1} \epsilon_1(s) - \epsilon_0(s)$$

We plot the spectral gap in the following graph (it appear that the gap is arround the 40th annealing step with a value of approximately 0.8):



Create a Jupyter notebook called qa_noiseless_simulation.
Instructions and questions :
- Code a simulation example for a small problem (<10 variables) and plot the five lowest eigenvalues (y axis) according to the annealing time steps (x axis).
- Plot the difference between your first lowest eigenvalue and the second lowest eigenvalue, this will display your spectral gap.
- Try to solve problems of different sizes (3-12 variables) with Ising instances generated from the same weight distribution.
- Do you observe any difference concerning the spectral gap ? In your opinion, how should scale the spectral gap value with the instance size (independently from the distribution of the weights) ?
- Take a single instance and create different tests where you rescale the weights of the Hamiltonian H_final. How is the spectral gap impacted ?
- In quantum annealers, the programming range of h_i and J_ij values is always finite (for example $h_i \in [-1, +1]$ and $h_{i,j} \in [-2, +2]$). If we need to rescale the values of one input problem to solve it with a quantum annealer, does it ease or worsen the resolution of the problem ? Explain why (take a look at the spectral gap) ?

TODO 6:
In practice, there is some uncertainty in the programming of D-Wave Hamiltonian (the electronic components do not always act ideally.
So instead of having the Hamiltonian :

$$H_{final} = \sum_{i=1}^{n} h_i \sigma_i^z + \sum_{i<j} J_{ij} \sigma_i^z \sigma_j^z$$

we have in practice the following Hamiltonian :

$$H_{final} = \sum_{i=1}^{n} (h_i + \delta_i) \sigma_i^z + \sum_{i<j} (J_{i,j} + \delta_{i,j}) \sigma_i^z \sigma_j^z$$

where δ_i and $\delta_{i,j}$ represent the uncertainty concerning the control of programming components.
In this experiment, we will consider that the noise varies at each time step and that the transverse field Hamiltonian H_{init} is noiseless and follows a Normal law centered at 0 with a small standard deviation (the standard deviation should be small compared to the weights of the Ising problem if you want a moderate profile of noise).

Create a Jupyter notebook called qa_noisy_simulation.
Instructions and questions :
- Code the noisy simulation and plot the spectral gap, reusing functions and schedules already defined above.
- Create plots with the five lowest eigenvalues considering the linear annealing schedule.
- At the beginning of the evolution, is the noise visible ? Why ?
- How does the noise affect the spectral gap in general ? (plot some examples and explain your thinking)
- Add some noise on the Hamiltonian H_{init} too.
- Is it better or worse ?
- Do you think the noise model included here is really representative of quantum noise ? How would you improve the noise model (please do not use chatGPT for this question). You can do some tests if needed.
- Considering that the annealing time depend on the spectral gap, what do you conclude when the noise profile gets stronger ?

TODO 7 (Bonus):
Create an Ising problem instance that contains a duplicated node with a strong couling to ensure that the variables representing a same node are strongly coupled (see for example slides 34 and 35). We will consider a unique constant strong coupling F_ϕ.

Create a Jupyter notebook called qa_topology_constraint.
Instructions and questions :
- Compare the spectral gap of the instance with the duplicated node and the same instance without duplication (meaning that the second instance as 1 variable less).
- In general, how would the spectral gap vary with the number of duplications ? Provide other tests if needed. Do not hesitate to write your thought process.
- If the coupling F_\phi is too strong to be programmed in the programming range defined by the quantum computer h_i and $J_{i,j}$ what can you do to program it anyway ? Does it ease or worsen the resolution of the problem ? Explain why ?